INS6021 · PHÁT TRIỂN PHẦN MỀM · FINAL PROJECT · NHÓM 1

e-HoSo

QUYẾT ĐỊNH KHO TỆP TƯƠNG THÍCH S3 THAY MINIO

So sánh phương án và quyết định (D17)

Phiên bản 1.0 · 28/10/2026

Người soạn và quyết định: Mai Thành Khang (Tech Lead) – bản nháp và quyết định do Claude thực hiện thay, theo uỷ quyền của PM
Người duyệt: Đoàn Quang Anh (Backend + CSDL)

Phiên bản	Ngày	Người thực hiện	Nội dung
0.1	28/10/2026	Mai Thành Khang (bản nháp do Claude hỗ trợ)	So sánh 4 phương án, đề xuất và kế hoạch kiểm chứng
1.0	28/10/2026	Mai Thành Khang (Claude chốt thay theo uỷ quyền của PM Đỗ Văn Tiến)	Đối chiếu SRS v1.1 (FR-08, NFR-04); ghi quyết định chọn Garage, điều kiện chuyển sang phương án dự phòng

1. Mục đích và phạm vi

Tài liệu so sánh các kho tệp tương thích S3 có thể thay MinIO để lưu tệp đính kèm hồ sơ (DR-05), và ghi quyết định của Trưởng kỹ thuật. Tài liệu xử lý vấn đề mở OI-05 của SRS v1.1 và GitHub issue #40.

Ngoài phạm vi: dịch vụ đám mây trả phí (AWS S3, Cloudflare R2…), vì NFR-08 yêu cầu toàn bộ hệ thống chạy cục bộ bằng một lệnh.

2. Bối cảnh

- SRS v1.0 và Project Plan chọn MinIO. SRS v1.1 (mục 2.4, 2.5, 9) đã đổi thành "kho tệp tương thích S3" và để Trưởng kỹ thuật chọn sản phẩm (OI-05).
- Tình trạng MinIO tính đến 10/2026 (nguồn ở mục 12):
  - 10/2025: ngừng phát hành image miễn phí, chuyển sang phân phối mã nguồn.
  - 2026: kho GitHub chuyển sang lưu trữ (archived), thông báo không còn bảo trì.
  - 09/2026: Docker Hub xoá `minio/minio`, `minio/mc`; quay.io từ chối tải ẩn danh.
- Hệ quả: `docker-compose.yml` của repo hiện chỉ có PostgreSQL và Mailpit, chưa có dịch vụ lưu tệp.

3. Nhu cầu lưu tệp của e-HoSo

#	Nhu cầu	Căn cứ	Mức
N1	Lưu, đọc, xoá tệp đính kèm (PutObject, GetObject, DeleteObject); siêu dữ liệu nằm trong PostgreSQL, kho tệp chỉ giữ nội dung theo khoá lưu trữ	DR-05	Bắt buộc
N2	Tệp chỉ tải được qua URL ký sẵn (presigned GET) có thời hạn 5 phút, sinh sau khi kiểm tra quyền	NFR-04	Bắt buộc
N3	Định địa chỉ kiểu path-style (`host/bucket/key`), không cần DNS	Chạy cục bộ bằng Docker	Bắt buộc
N4	Chữ ký SigV4, dùng được AWS SDK for Java v2	Stack Spring Boot	Bắt buộc
N5	Tải lên tệp tối đa 10 MB (mỗi hồ sơ tối đa 30 MB) bằng một lệnh PutObject; tệp 10 MB hoàn tất dưới 10 giây	FR-08, NFR-01	Bắt buộc; không cần multipart
N6	Khởi tạo bucket và khoá truy cập tự động, không thao tác tay	NFR-08	Bắt buộc
N7	Không mất tệp khi khởi động lại dịch vụ; xoá được toàn bộ dữ liệu demo bằng lệnh khởi tạo lại	NFR-05, SRS mục 8	Bắt buộc

#	Nhu cầu	Căn cứ	Mức
N8	Tổng tài nguyên toàn hệ thống vừa máy 8 GB RAM	NFR-08	Bắt buộc

Nhật ký chỉ được thêm (FR-23, DR-06) lưu trong PostgreSQL. Vì vậy kho tệp không cần versioning hay Object Lock.

4. Tiêu chí đánh giá

Mã	Tiêu chí	Trọng số	Ghi chú
TC1	Phân phối và bảo trì: có image chính thức tải ẩn danh, ghim được phiên bản, còn vá lỗi	25%	Tiêu chí loại: điểm 1 thì loại
TC2	Tài nguyên, phù hợp NFR-08	20%	Đo lại khi kiểm chứng
TC3	Độ phủ S3 cho N1–N5	20%	
TC4	Khởi tạo một lệnh, cấu hình đơn giản (N6)	15%	
TC5	Độ trưởng thành, cộng đồng	10%	
TC6	Hỗ trợ demo và quan sát (giao diện web)	5%	
TC7	Giấy phép	5%	

5. Các phương án

A. SeaweedFS (Go, Apache 2.0). Hệ lưu trữ phân tán gồm master, volume, filer và cổng S3, chạy được trong một container bằng `weed server -s3` hoặc `weed mini`. Image `chrislusf/seaweedfs` có bản amd64 và arm64. Trưởng thành, nhiều tính năng ngoài S3. Chỉ mục vị trí đối tượng nằm trong bộ nhớ, nên RAM tăng theo số đối tượng; ở quy mô dữ liệu giả lập của e-HoSo, ảnh hưởng này không đáng kể.

B. Garage (Rust, AGPL v3). Kho đối tượng gọn nhẹ, một tệp thực thi, dùng trong sản xuất từ 2020. Image chính thức `dxflrs/garage` (v2.4.1, 09/2026). Hỗ trợ SigV4, path-style, presigned URL, multipart. Không có ACL, bucket policy, versioning; quyền truy cập theo cặp khoá–bucket. Không có giao diện web. Từ v2.3 có chế độ `--single-node` tự tạo layout khi khởi động lần đầu.

C. RustFS (Rust, Apache 2.0). Thay thế MinIO có cách dùng gần giống MinIO (API cổng 9000, console 9001). Bản 1.0.0 GA ngày 16/09/2026, độ phủ S3 rộng, có tài liệu chính thức cho AWS SDK Java v2. Mới GA khoảng 6 tuần; một số tính năng tương thích còn ở mức preview.

D. Tự build MinIO từ mã nguồn (Go, AGPL v3). Giữ nguyên tên sản phẩm của Project Plan, nhưng thượng nguồn đã ngừng bảo trì nên không còn bản vá bảo mật. Image do bên thứ ba build lại (Chainguard, fork `pgsty/silo`) không phải nguồn chính thức. Thêm bước build vào quy trình, trái với tinh thần "một lệnh" của NFR-08.

6. Bảng so sánh

Tiêu chí	A. SeaweedFS	B. Garage	C. RustFS	D. MinIO tự build
Image chính thức tải ẩn danh	Có	Có	Có	Không
Còn bảo trì	Có (4.48, 09/2026)	Có (v2.4.1, 09/2026)	Có (1.0.0, 09/2026)	Không (archived)
Giấy phép	Apache 2.0	AGPL v3	Apache 2.0	AGPL v3
SigV4, path-style, presigned (N2–N4)	Có	Có	Có	Có
ACL, bucket policy, versioning	Một phần	Không	Có	Có
Tiến trình trong container	Một tiến trình, nhiều thành phần	Một	Một	Một
Giao diện web	Có	Không	Có	Đã bị lược bớt
RAM khi rỗi (số liệu cộng đồng, không đồng nhất)	~256 MB	~40–128 MB	Chưa có số đo tin cậy	~512 MB
Rủi ro chính	Nhiều thành phần; cấu hình khoá qua `weed shell` hoặc tệp cấu hình	Không có UI; cần tệp `garage.toml` và bước tạo khoá	Mới GA, ít thời gian kiểm chứng thực tế	Không có bản vá bảo mật

Số liệu RAM lấy từ bài viết cộng đồng, đo trong điều kiện khác nhau; chỉ dùng để xếp hạng sơ bộ. Số đo dùng trong SDD lấy từ kiểm chứng ở mục 9.

7. Chấm điểm (thang 1–5, đánh giá định tính)

Tiêu chí	Trọng số	A	B	C	D
TC1 Phân phối, bảo trì	25%	5	5	5	1 (loại)
TC2 Tài nguyên	20%	4	5	3	4
TC3 Độ phủ S3	20%	4	4	5	5
TC4 Khởi tạo một lệnh	15%	3	4	5	2
TC5 Độ trưởng thành	10%	5	4	2	3
TC6 Demo, quan sát	5%	4	2	5	3
TC7 Giấy phép	5%	5	3	5	3
Tổng có trọng số		**4,25**	**4,30**	**4,30**	Loại

Nhận xét: - D bị loại ở TC1. - B và C bằng điểm, A sát sau; chênh lệch nằm trong sai số của cách chấm định tính. Mục 8 nêu cách phân định. - Về giấy phép: e-HoSo chỉ chạy Garage như một dịch vụ riêng, không sửa mã và không phân phối kèm sản phẩm, nên AGPL không tạo nghĩa vụ đáng kể trong phạm vi đồ án. Đây là nhận định kỹ thuật, không phải tư vấn pháp lý.

8. Phân tích và đề xuất

Khi tổng điểm bằng nhau, ưu tiên phương án tốt hơn ở tiêu chí gắn với yêu cầu mức Must và rủi ro cho buổi demo:

Yếu tố phân định	B. Garage	C. RustFS
NFR-08 (Must), TC2 trọng số 20%	Nhẹ nhất trong các phương án	Chưa có số đo tin cậy
Rủi ro demo (R4), TC5	Chạy sản xuất từ 2020	GA từ 16/09/2026
N1–N8	Đáp ứng đủ; phần thiếu (ACL, policy, versioning) không thuộc nhu cầu	Đáp ứng đủ
Ưu thế của C	–	Khởi tạo đơn giản hơn, có console web (TC4, TC6: tổng trọng số 20%)

Ưu thế của RustFS khắc phục được ở mức triển khai: bước tạo khoá và bucket của Garage được tự động hoá trong Compose (N6); khi demo, xem tệp qua chính giao diện e-HoSo. Ngược lại, rủi ro về độ trưởng thành của RustFS không khắc phục được trong 6 tuần của dự án.

Đề xuất: B. Garage. Dự phòng: C. RustFS.

Nguyên tắc thiết kế áp dụng cho mọi phương án (đưa vào SDD, D11): 1. Backend truy cập kho tệp qua một giao diện riêng (ví dụ `FileStoragePort`) và một lớp cài đặt dùng AWS SDK for Java v2; không gọi API riêng của sản phẩm. 2. Cấu hình qua biến môi trường: endpoint nội bộ, endpoint công khai, region, access key, secret key, tên bucket; bật `forcePathStyle(true)` cho cả `S3Client` và `S3Presigner`. 3. `S3Presigner` ký theo **endpoint công khai** (trình duyệt truy cập được), vì host là một phần của chữ ký; `S3Client` dùng endpoint nội bộ trong mạng Compose. 4. URL ký sẵn có thời hạn 5 phút và chỉ được sinh sau khi kiểm tra quyền theo ma trận phân quyền (NFR-04). 5. Kiểm tra dung lượng (10 MB/tệp, 30 MB/hồ sơ) và định dạng theo nội dung tệp ở backend trước khi ghi vào kho (FR-08). 6. Hệ quả: đổi sang phương án dự phòng chỉ cần sửa `docker-compose.yml` và biến môi trường, không sửa mã nghiệp vụ.

9. Kiểm chứng khi tích hợp vào repo (giới hạn 2 giờ)

Thực hiện khi thêm dịch vụ lưu tệp vào `docker-compose.yml` (mục 11, việc 2), chạy cùng PostgreSQL và Mailpit hiện có. Kết quả để trống cho người thực hiện điền; không ghi kết quả khi chưa chạy thật.

#	Bước kiểm chứng	Đạt khi	Kết quả
K1	`docker compose up -d` từ bản clone mới, không thao tác thêm	Dịch vụ sẵn sàng; bucket và khoá có sẵn	?
K2	`docker stats` sau 5 phút rỗi và sau khi tải lên 100 tệp mẫu	Ghi số đo; toàn hệ thống còn dư trong 8 GB	?
K3	Từ Spring Boot (AWS SDK v2): put, get, delete một tệp PDF mẫu	Thành công	?

#	Bước kiểm chứng	Đạt khi	Kết quả
K4	Sinh presigned GET 5 phút theo endpoint công khai, tải bằng trình duyệt; thử lại sau khi hết hạn	Tải được trong hạn, bị từ chối khi hết hạn	?
K5	Tải lên tệp 10 MB qua backend	Thành công, dưới 10 giây trên máy cục bộ	?
K6	`docker compose restart`, đọc lại tệp đã tải lên; `docker compose down -v` rồi `up`	Tệp còn nguyên sau restart; dữ liệu sạch sau khi khởi tạo lại	?
K7	Chạy trên máy Apple Silicon (arm64) và máy x86 của nhóm	Cả hai chạy được	?

Người thực hiện: ? · Ngày: ? · Kết luận: ?

Cấu hình nháp cho Garage (chưa chạy thử; xác nhận ở K1):

```yaml

# docker-compose.yml – bổ sung dịch vụ

storage: image: dxflrs/garage:v2.4.1 command: ["/garage", "server", "--single-node"] restart: unless-stopped environment: GARAGE_RPC_SECRET: ${GARAGE_RPC_SECRET} volumes: - ./infra/garage.toml:/etc/garage.toml:ro - garage-meta:/var/lib/garage/meta - garage-data:/var/lib/garage/data ports: - "3900:3900" # S3 API ```

```toml

# infra/garage.toml

metadata_dir = "/var/lib/garage/meta" data_dir = "/var/lib/garage/data" db_engine = "sqlite" replication_factor = 1 rpc_bind_addr = "[::]:3901"

[s3_api] s3_region = "garage" api_bind_addr = "[::]:3900" ```
```

Cần xác nhận ở K1: cách tạo khoá truy cập và bucket tự động (tuỳ chọn tạo bucket mặc định của `--single-node` nếu phiên bản hỗ trợ, hoặc một container khởi tạo chạy `garage key create` và `garage bucket create` một lần); region trong cấu hình SDK phải trùng `s3_region`.

10. Quyết định

Mục	Nội dung
Phương án chọn	**B. Garage** (`dxflrs/garage`, ghim phiên bản v2.4.1)
Phương án dự phòng	C. RustFS (`rustfs/rustfs`, ghim phiên bản 1.0.0)
Lý do	Đáp ứng đủ N1–N8; tốt nhất ở NFR-08 (Must) và độ trưởng thành, giảm rủi ro demo (R4); các tính năng còn thiếu không thuộc nhu cầu của e-HoSo (mục 8)
```

Mục	Nội dung
Điều kiện chuyển sang dự phòng	Một trong các bước K1, K3, K4, K6, K7 không đạt và không khắc phục được trong giới hạn 2 giờ kiểm chứng. Khi chuyển, cập nhật tài liệu này lên v1.1
Phạm vi ảnh hưởng	`docker-compose.yml`, cấu hình backend, SDD (D11), sơ đồ deployment (D12), Test Plan (D16, ca K1–K7). Không đổi yêu cầu trong SRS
Ngày quyết định	28/10/2026
Người quyết định	Mai Thành Khang (Tech Lead). Claude chốt thay theo uỷ quyền của PM Đỗ Văn Tiến; Khang xác nhận tại họp tuần 01/11/2026
Người duyệt	Đoàn Quang Anh (Backend + CSDL): ?

11. Việc cần làm sau quyết định

#	Việc	Người làm	Nơi làm
1	Khang xác nhận quyết định tại họp 01/11; Quang Anh duyệt tài liệu này	Khang, Quang Anh	Họp tuần 01/11
2	Thêm dịch vụ `storage` vào `docker-compose.yml`, chạy kiểm chứng K1–K7, ghi ADR vào `docs/design/`, đóng issue #40 khi PR merge	Khang (Quang Anh duyệt PR)	Claude Code, repo e-hoso
3	Ghi tên sản phẩm đã chọn và đóng OI-05 ở lần sửa SRS kế tiếp (v1.2). Không cần yêu cầu thay đổi vì nội dung yêu cầu không đổi	Tiến	Thư mục dự án
4	Cập nhật Project Plan: công nghệ (thay MinIO bằng Garage), rủi ro liên quan	Tiến	Thư mục dự án
5	Đưa nguyên tắc ở mục 8 vào SDD v1.0 (D11) và sơ đồ deployment (D12)	Khang	Bước 2 của M2

12. Tài liệu tham chiếu

- e-HoSo, Đặc tả yêu cầu phần mềm v1.1: mục 2.4, 2.5, 8 (DR-05), 9; FR-08; NFR-01, NFR-04, NFR-05, NFR-08; OI-05
- GitHub issue #40, #37 – https://github.com/cachuabi/e-hoso/issues/40 , https://github.com/cachuabi/e-hoso/issues/37
- Gigazine, MinIO ngừng phát hành Docker image miễn phí (10/2025) – https://gigazine.net/gsc_news/en/20251023-minio-stops-distributing-free-docker-images
- RAGFlow issue #13840, tình trạng MinIO community edition – https://github.com/infiniflow/ragflow/issues/13840
- Jacar, sự cố tải image MinIO (09/2026) – https://jacar.es/en/fix-pull-access-denied-minio-docker/
- Jacar, chuyển từ MinIO sang Garage bằng Docker – https://jacar.es/en/migrate-minio-to-garage-with-docker/
- Garage, S3 compatibility status – https://garagehq.deuxfleurs.fr/documentation/reference-manual/s3-compatibility
- RamNode, triển khai Garage single-node – https://ramnode.com/guides/garage
- Linuxiac, RustFS 1.0 GA – https://linuxiac.com/rustfs-1-0-s3-compatible-object-storage-reaches-ga/
- AICYBR, RustFS 1.0 GA – https://aicybr.com/blog/rustfs-1-0-ga-s3-object-storage
- RustFS, Java SDK guide – https://docs.rustfs.com/en/developer/sdk/java

- SeaweedFS, README và Getting Started – https://github.com/chrislusf/seaweedfs/
- HS5 wiki, nhận xét về bộ nhớ của SeaweedFS – https://github.com/uroni/hs5/wiki
- Glukhov, Self-Hosted S3 Alternatives in 2026 – https://glukhov.org/data-infrastructure/object-storage/self-hosted-s3-alternatives/
- Elestio, RustFS vs SeaweedFS vs Garage – https://blog.elest.io/rustfs-vs-seaweedfs-vs-garage-which-minio-alternative-should-you-pick/
- Pistack, SeaweedFS vs MinIO vs Garage (số liệu RAM) – https://www.pistack.xyz/posts/seaweedfs-vs-minio-vs-garage/